

Resumo de Defesa - Projeto Tramos

1. Visao geral do projeto

O Tramos e uma plataforma simples para conectar empresas que precisam de trabalhadores temporarios com freelancers em busca de oportunidades rapidas.

Na versao atual, o sistema foi migrado para uma arquitetura **server-side** usando:

- **FastAPI** no backend
- **Jinja** para renderizacao de HTML no servidor
- **MySQL** como banco de dados
- **PyMySQL** para executar SQL direto
- **SessionMiddleware** para login em sessao

O projeto deixou de depender de um frontend separado consumindo API para as telas principais. Agora, o proprio backend entrega as paginas prontas.

2. O que e Jinja

Jinja e um **motor de templates para Python**. Ele permite misturar HTML com dados vindos do backend.

Em vez de montar a tela com JavaScript no navegador, o servidor ja monta o HTML e devolve a pagina pronta.

Sintaxe principal do Jinja

- `{{ variavel }}`: imprime valor na tela
- `{% ... %}`: executa logica como `if`, `for`, `include`, `extends`
- `{# comentario #}`: comentario do template

Exemplos simples

- `{{ user.nome }}` mostra o nome do usuario
- `{% if user %}` permite exhibir conteudo so se estiver logado
- `{% for vaga in vagas %}` repete um bloco para cada vaga

Como o Jinja aparece neste projeto

- `templates/base.html`: layout principal
- `templates/index.html`: home com listagem de vagas
- `templates/profile.html`: pagina de perfil

- `templates/partials/nav.html`: navbar compartilhada
- `templates/partials/flash.html`: mensagens server-side

Ou seja: o HTML é montado com dados reais vindos do Python e do banco.

3. O que significa dizer que o projeto é server-side

Significa que a maior parte do processamento de tela acontece no servidor.

Fluxo resumido:

1. O navegador faz uma requisição HTTP.
2. O FastAPI recebe a requisição.
3. O backend consulta o banco, valida sessão e prepara os dados.
4. O Jinja combina os dados com o template HTML.
5. O servidor devolve a página pronta para o navegador.

Diferença para client-side

Client-side

- o navegador recebe HTML mais vazio
- JavaScript busca dados depois
- a tela é montada no browser

Server-side

- o navegador recebe HTML já pronto
- menos dependência de JavaScript
- login, permissões e mensagens ficam mais centralizados

Neste projeto, a escolha por server-side fez sentido porque:

- segue o modelo base do professor
- simplifica formulários e redirecionamentos
- facilita controle de sessão e autorização
- reduz complexidade no frontend
- deixa a aplicação mais coerente para uma defesa acadêmica

4. Por que usar Jinja neste projeto

Jinja foi usado porque ele resolve muito bem páginas com:

- listagem de registros

- formulários
- condições de exibição
- reaproveitamento de layout
- renderização baseada em sessão

Exemplos reais daqui:

- mostrar botões diferentes para empresa e freelancer na home
- esconder ações protegidas quando o usuário não está logado
- exibir avatar ou inicial do nome
- repetir cards de vagas e candidaturas
- mostrar mensagens de sucesso ou erro sem depender de JavaScript

5. Estrutura do projeto

Backend

- `tramos/backend/app/main.py`
- arquivo central da aplicação
- define rotas
- valida formulários
- controla sessão
- faz consultas SQL
- gerencia avatar, perfil, vagas e candidaturas
- `tramos/backend/app/database.py`
- lê a `DATABASE\_URL`
- abre conexão PyMySQL
- concentra configuração de banco e segredo de sessão
- `tramos/backend/templates/`
- contém os templates Jinja
- `tramos/backend/static/`
- CSS e imagens
- `tramos/backend/tramos.sql`
- cria as tabelas e insere dados fake
- `tramos/backend/seed.py`
- recria o banco a partir do SQL

Tabelas principais

- `Usuario`
 - nome, email, senha hash, tipo, avatar, avatar_mime
- `Categoria`
 - categorias de vagas
- `Vaga`
 - titulo, descricao, data, local, pagamento, empresa responsavel
- `Candidatura`
 - liga freelancer a vaga e guarda status

6. Como o projeto funciona na pratica

6.1 Home `/`

Na home, o backend busca vagas no banco, aplica filtros e envia tudo para `index.html`.

O template:

- lista vagas
- mostra categoria, local, data e pagamento
- diferencia visualmente empresa e freelancer quando estao logados

6.2 Login `/login`

No login:

1. o usuario envia email e senha
2. o backend busca o usuario pelo email
3. a senha digitada e comparada com o hash bcrypt
4. se estiver correta, os dados do usuario entram na sessao
5. o sistema redireciona para a home

6.3 Cadastro `/cadastro`

O cadastro:

- valida nome, email, senha e tipo
- impede email duplicado
- salva a senha com bcrypt

- cria sessao logo apos o cadastro

6.4 Perfil `/perfil`

Na pagina de perfil o usuario consegue:

- ver seus dados
- editar nome, email e senha
- enviar avatar
- acompanhar candidaturas ou vagas publicadas

6.5 Empresa publicando vaga

Quando o usuario e empresa, ele pode:

- abrir `/vaga/nova`
- preencher formulario
- salvar a vaga no banco
- depois visualizar candidatos

6.6 Freelancer se candidatando

Quando o usuario e freelancer, ele pode:

- abrir a vaga
- clicar em candidatar-se
- gerar um registro em `Candidatura`
- acompanhar o status no perfil

6.7 Empresa avaliando candidaturas

A empresa pode:

- acessar a vaga publicada
- ver a lista de candidatos
- aceitar ou recusar

O status fica salvo no banco e aparece depois para o freelancer.

7. Como a autenticacao e autorizacao funcionam

Autenticacao

Autenticacao responde: ****quem e o usuario?****

No projeto, isso e feito com:

- `SessionMiddleware`
- cookie de sessao
- dados do usuario guardados em `request.session`

Autorizacao

Autorizacao responde: ****o que esse usuario pode fazer?****

No projeto, isso e feito com:

- verificacao se ha usuario logado
- verificacao do tipo do usuario: `empresa` ou `freelancer`

Funcoes importantes

- `current\_user(request)` : le o usuario da sessao
- `sync\_session\_user(request)` : revalida o usuario no banco a cada request
- `require\_user(request, role=None)` : exige login e, se necessario, tipo especifico

Por que isso foi importante

Antes, existia um problema visual de reaproveitamento de acesso depois de logout em abas e cache. Para reduzir isso:

- a sessao passou a ser revalidada no servidor
- respostas receberam cabecalhos de `no-cache`
- logout passou a limpar cache do navegador

Assim, o controle ficou muito mais confiavel.

8. Por que nao usar SQLAlchemy

O projeto foi implementado sem SQLAlchemy por dois motivos principais:

1. era uma exigencia do trabalho seguir o modelo sem ORM
2. para esse tamanho de projeto, SQL direto deixou o funcionamento mais explicito

Vantagens do SQL direto aqui

- mostra claramente o que esta sendo consultado ou inserido
- facilita explicar cada query na defesa

- reduz camada extra de abstracao
- aproxima o projeto do modelo usado como referencia

Como ficou

Em vez de models ORM, o projeto usa:

- ``query_one(...)``
- ``query_all(...)``
- funcoes especificas para buscar usuario, vagas, candidaturas e perfil

Ou seja: a regra de negocio continua no Python, mas o acesso ao banco e feito por SQL puro.

9. Como o avatar funciona

O avatar foi implementado no padrao pedido:

- upload feito por formulario
- arquivo validado no backend
- tipo identificado pelos bytes do arquivo
- imagem salva direto no MySQL em ``LONGBLOB``
- MIME salvo em ``avatar_mime``

Na hora de mostrar:

1. o backend le os bytes do banco
2. converte para base64
3. monta uma ``data URL``
4. o template coloca isso no ``src`` da imagem

Por que isso funciona bem

- evita depender de pasta de uploads
- deixa tudo centralizado no banco
- segue o modelo do projeto base citado

10. O que foi migrado em relacao ao projeto original

O projeto original tinha uma ideia mais proxima de:

- frontend separado
- mais dependencia de JavaScript para interacao
- consumo de API para montar partes da interface

Depois da migracao, passou a ter:

- telas renderizadas no servidor
- formularios HTML comuns
- padrao POST/Redirect/GET
- sessao no backend
- SQL direto com PyMySQL
- mensagens exibidas pelo servidor
- confirmacoes em paginas server-side

Hoje ainda existe JavaScript minimo de interface, como:

- abrir o menu mobile
- alternar visibilidade da senha

Mas as ****mensagens e os fluxos principais**** nao dependem de JavaScript.

11. Por que a arquitetura server-side foi uma boa decisao

1. Simplicidade

Com server-side, o fluxo fica mais direto:

- formulario envia dados
- backend processa
- backend redireciona
- pagina volta pronta

2. Seguranca e controle

Permissoes e sessao ficam centralizadas no backend.

3. Coerencia com o trabalho

A arquitetura ficou alinhada ao padrao do projeto de referencia usado na disciplina.

4. Melhor defesa de autoria

Fica mais facil explicar:

- onde a sessao e criada
- onde a permissao e validada

- onde a query roda
- onde o HTML final é montado

12. Arquivos que voce deve saber explicar na defesa

`app/main.py`

Se o professor abrir esse arquivo, voce deve explicar que ele concentra:

- criacao do app FastAPI
- configuracao do Jinja
- filtros e globais dos templates
- funcoes de autenticacao e autorizacao
- acesso ao banco com SQL
- rotas principais

`app/database.py`

Explique que esse arquivo:

- lê o `.env`
- interpreta a `DATABASE\_URL`
- monta a conexao com MySQL

`templates/base.html`

Explique que é o layout base e que as outras paginas usam `extends`.

`templates/index.html`

Explique que a home usa dados dinamicos do backend e adapta a interface conforme o tipo do usuario.

`trampos.sql`

Explique que é a base do banco e que as tabelas principais são Usuario, Categoria, Vaga e Candidatura.

13. Resposta curta pronta para abrir a defesa

"Esse projeto foi migrado para uma arquitetura server-side com FastAPI e Jinja. Em vez de depender de um frontend separado para montar a interface, o backend agora busca os dados no MySQL, valida a sessao e renderiza as paginas HTML no servidor. Eu tambem removi o SQLAlchemy e passei a usar PyMySQL com SQL direto, seguindo o modelo de

referencia. Assim o sistema ficou mais simples de explicar, com melhor controle de autenticação, autorização, formulários, mensagens e upload de avatar."

14. Perguntas e respostas para uma defesa de autoria

1. O que é Jinja?

Jinja é um motor de templates do Python. Ele permite gerar HTML dinâmico no servidor usando variáveis, condicionais e repetições.

2. Onde o Jinja está sendo usado no seu projeto?

Ele está nos arquivos da pasta `templates/`. O FastAPI envia dados para esses templates e o Jinja monta a página final.

3. Qual a principal diferença entre server-side e client-side?

No client-side, o navegador monta a interface com JavaScript depois de buscar dados. No server-side, o servidor já monta a tela e entrega o HTML pronto.

4. Por que você escolheu server-side?

Porque simplifica os fluxos com formulário, melhora o controle de sessão, reduz dependência de JavaScript e segue o padrão do projeto base usado como referência.

5. Por que usar FastAPI se o projeto é server-side?

Porque FastAPI não serve apenas para API JSON. Ele também pode renderizar templates, receber formulários, controlar sessão e servir arquivos estáticos.

6. Como o login funciona tecnicamente?

O formulário envia e-mail e senha. O backend busca o usuário no banco, compara a senha com bcrypt e salva o usuário na sessão se estiver correta.

7. Onde a sessão fica?

A sessão é controlada pelo `SessionMiddleware`. O navegador guarda o cookie e o backend lê os dados da sessão a cada requisição.

8. Como você garante que o usuário ainda existe e tem permissão?

Eu uso `sync\_session\_user()` para revalidar o usuário no banco e `require\_user()` para bloquear acesso sem login ou com tipo errado.

9. Qual a diferença entre autenticação e autorização no seu sistema?

Autenticacao e identificar quem esta logado. Autorizacao e decidir o que esse usuario pode fazer, por exemplo permitir publicar vaga apenas para empresa.

10. Como uma empresa publica uma vaga?

A empresa acessa `/vaga/nova``, envia o formulario e o backend salva os dados na tabela ``Vaga``.

11. Como um freelancer se candidata?

Ele entra no detalhe da vaga e faz um POST para a rota de candidatura. Isso cria um registro na tabela ``Candidatura``.

12. Como a empresa aceita ou recusa um candidato?

Na tela da vaga ela envia uma acao para a rota de status da candidatura, e o backend atualiza o campo ``status`` no banco.

13. Por que voce nao usou SQLAlchemy?

Porque o trabalho pedia um modelo sem ORM e, neste caso, SQL direto deixou a logica mais explicita e mais facil de defender.

14. Como o acesso ao banco foi implementado?

Com PyMySQL. O arquivo ``database.py`` monta a conexao e o ``main.py`` executa queries SQL diretas.

15. Quais sao as principais tabelas?

``Usuario``, ``Categoria``, ``Vaga`` e ``Candidatura``.

16. Como o avatar funciona?

O usuario envia a imagem por formulario, o backend valida tamanho e tipo, salva os bytes em ``LONGBLOB`` no MySQL e depois converte para base64 para mostrar na tela.

17. Por que salvar avatar no banco em vez de pasta?

Porque esse foi o padrao solicitado e porque centraliza os dados. O banco passa a guardar tanto as informacoes do usuario quanto a imagem dele.

18. Como as mensagens aparecem sem depender de JavaScript?

O backend salva uma flash message na sessao, redireciona e o template renderiza a mensagem na pagina seguinte.

19. O que e POST/Redirect/GET e por que voce usou?

E um padrao em que o formulario faz POST, o backend processa e depois redireciona com GET. Isso evita reenvio acidental ao atualizar a pagina e deixa a navegacao mais limpa.

20. Como voce melhorou o problema de acesso depois de logout?

Eu revalidei a sessao no servidor, adicionei cabecalhos de no-cache e fiz o logout limpar cache para evitar reaproveitamento visual de paginas protegidas.

21. Se o professor perguntar "onde esta a logica principal?", o que responder?

No `app/main.py`. E ali que ficam rotas, validacoes, sessao, autorizacao e a maior parte das queries.

22. Se ele perguntar "como provar que o HTML nao esta fixo?", o que responder?

Eu mostro os `{ { ... } }` e `{ % ... % }` dos templates Jinja e explico que os dados mudam conforme sessao, usuario, vagas e candidaturas retornadas pelo banco.

23. Se ele perguntar "o que ainda usa JavaScript?", o que responder?

Apenas comportamentos pequenos de interface, como abrir menu mobile e mostrar ou ocultar senha. As mensagens e os fluxos principais estao no servidor.

24. Se ele perguntar "qual foi a maior mudanca do projeto?", o que responder?

Foi sair de uma estrutura mais separada e orientada a frontend para uma aplicacao server-side unificada, com sessao no backend, templates Jinja e SQL direto.

25. Se ele perguntar "por que essa arquitetura e adequada para esse trabalho?", o que responder?

Porque ela entrega todas as funcionalidades exigidas com menos complexidade, melhor controle de autenticacao e maior facilidade de manutencao e explicacao.

15. Perguntas um pouco mais exigentes

"Por que o servidor precisa renderizar a pagina se eu poderia usar fetch?"

Porque neste projeto a tela depende muito de sessao, permissao, feedback de formulario e renderizacao simples de dados. Server-side resolve isso com menos camadas.

"Se FastAPI e famoso por API, por que usar com Jinja?"

Porque ele continua sendo um framework web completo para rotas HTTP. Ele pode responder JSON, HTML, arquivos estaticos e formularios.

"SQL direto nao fica pior?"

Depende do tamanho do sistema. Aqui ele ficou bom porque o dominio e pequeno e as consultas ficam claras, curtas e faceis de auditar.

"Como voce separou visual e logica?"

A logica fica no Python e os templates ficam em HTML com Jinja. O CSS esta separado em ``static/css/styles.css``.

16. Como responder com seguranca sem decorar demais

Se voce travar, siga esta linha:

1. diga primeiro o objetivo da tela
2. diga qual rota atende essa tela
3. diga de onde os dados vem
4. diga como o template monta o HTML
5. diga qual regra de permissao existe ali

Exemplo:

"Na home, a rota ``/`` busca vagas no MySQL, aplica filtros, envia os dados para o template ``index.html`` e o Jinja renderiza os cards. Se o usuario estiver logado, a interface muda um pouco conforme o tipo dele."

17. Resumo final em uma frase

O Trampos e uma aplicacao web server-side feita com FastAPI, Jinja e MySQL, na qual o backend controla sessao, permissao, formularios, consultas SQL e renderizacao das paginas dinamicas.